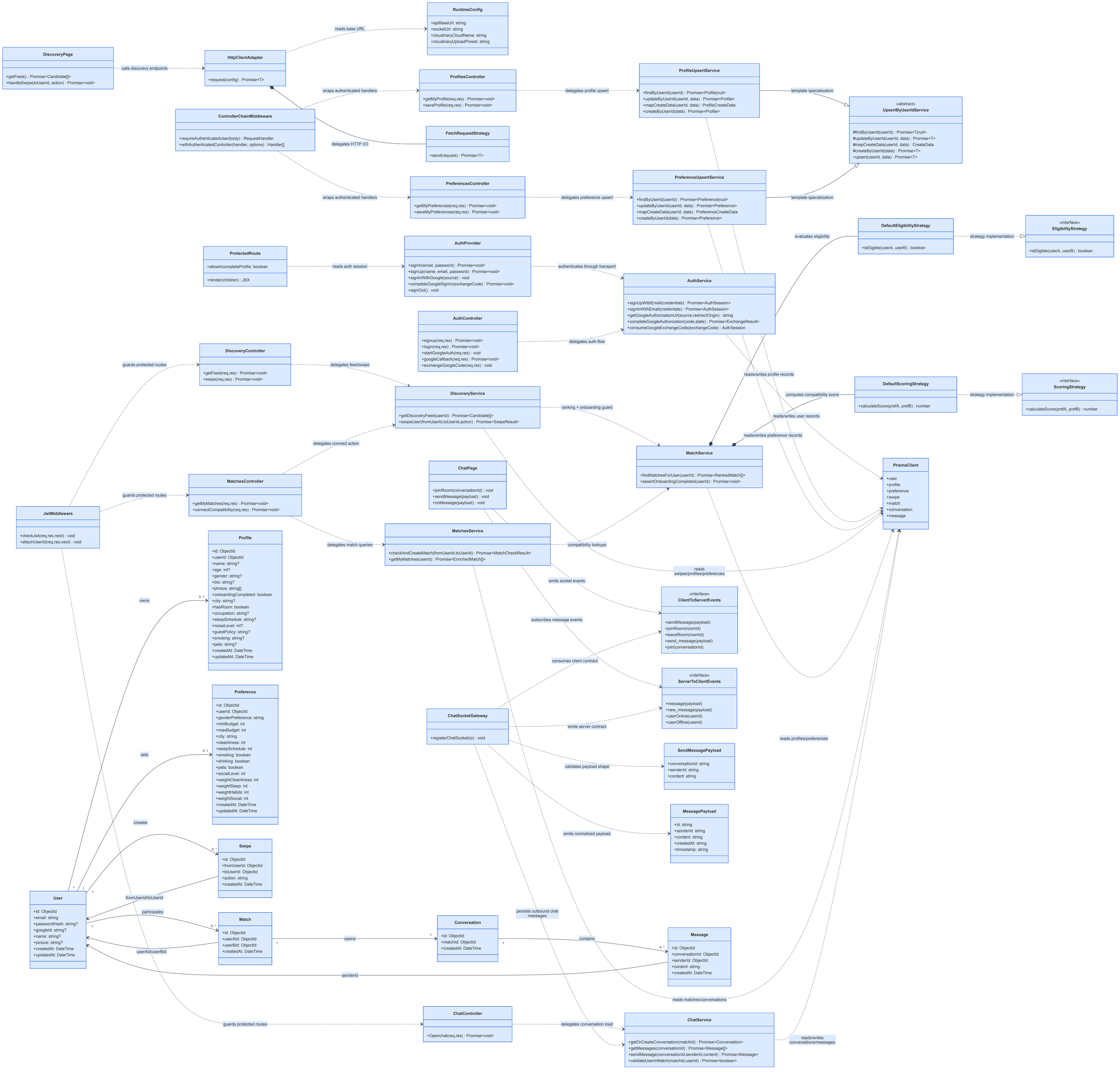
RuntimeConfig
+apiBaseUrl: string
+socketUrl: string
+cloudinaryCloudName: string
+cloudinaryUploadPreset: string
reads base URL
DiscoveryPage
+getFeed() : Promise<Candidate[]>
+handleSwipe(toUserId, action) : Promise<void>
calls discovery endpoints
HttpClientAdapter
+request(config) : Promise<T>
wraps authenticated handlers
ProfilesController
+getMyProfile(req,res) : Promise<void>
+saveProfile(req,res) : Promise<void>
delegates profile upsert
ProfileUpsertService
+findByUserId(userId) : Promise<Profile|null>
+updateByUserId(userId, data) : Promise<Profile>
+mapCreateData(userId, data) : ProfileCreateData
+createByUserId(data) : Promise<Profile>
template specialization
«abstract»
UpsertByUserIdService
#findByUserId(userId) : Promise<T|null>
#updateByUserId(userId, data) : Promise<T>
#mapCreateData(userId, data) : CreateData
#createByUserId(data) : Promise<T>
+upsert(userId, data) : Promise<T>
ControllerChainMiddleware
+requireAuthenticatedUser(body) : RequestHandler
+withAuthenticatedController(handler, options) : Handler[]
delegates HTTP I/O
FetchRequestStrategy
+send(request) : Promise<T>
wraps authenticated handlers
PreferencesController
+getMyPreferences(req,res) : Promise<void>
+saveMyPreferences(req,res) : Promise<void>
delegates preference upsert
PreferenceUpsertService
+findByUserId(userId) : Promise<Preference|null>
+updateByUserId(userId, data) : Promise<Preference>
+mapCreateData(userId, data) : PreferenceCreateData
+createByUserId(data) : Promise<Preference>
template specialization
evaluates eligibility
DefaultEligibilityStrategy
+isEligible(userA, userB) : boolean
strategy implementation
«interface»
EligibilityStrategy
+isEligible(userA, userB) : boolean
ProtectedRoute
+allowIncompleteProfile: boolean
+render(children) : JSX
reads auth session
AuthProvider
+signIn(email, password) : Promise<void>
+signUp(name, email, password) : Promise<void>
+signInWithGoogle(source) : void
+completeGoogleSignIn(exchangeCode) : Promise<void>
+signOut() : void
authenticates through transport
AuthService
+signUpWithEmail(credentials) : Promise<AuthSession>
+signInWithEmail(credentials) : Promise<AuthSession>
+getGoogleAuthorizationUrl(source,redirectOrigin) : string
+completeGoogleAuthorization(code,state) : Promise<ExchangeResult>
+consumeGoogleExchangeCode(exchangeCode) : AuthSession
AuthController
+signup(req,res) : Promise<void>
+login(req,res) : Promise<void>
+startGoogleAuth(req,res) : void
+googleCallback(req,res) : Promise<void>
+exchangeGoogleCode(req,res) : void
delegates auth flow
guards protected routes
DiscoveryController
+getFeed(req,res) : Promise<void>
+swipe(req,res) : Promise<void>
delegates feed/swipe
DiscoveryService
+getDiscoveryFeed(userId) : Promise<Candidate[]>
+swipeUser(fromUserId,toUserId,action) : Promise<SwipeResult>
ranking + onboarding guard
reads/writes profile records
computes compatibility score
DefaultScoringStrategy
+calculateScore(prefA, prefB) : number
strategy implementation
«interface»
ScoringStrategy
+calculateScore(prefA, prefB) : number
reads/writes user records
reads/writes preference records
delegates connect action
MatchService
+findMatchesForUser(userId) : Promise<RankedMatch[]>
+assertOnboardingCompleted(userId) : Promise<void>
ChatPage
+joinRoom(conversationId) : void
+sendMessage(payload) : void
+onMessage(payload) : void
PrismaClient
+user
+profile
+preference
+swipe
+match
+conversation
+message
guards protected routes
MatchesController
+getMyMatches(req,res) : Promise<void>
+connectCompatibility(req,res) : Promise<void>
JwtMiddleware
+checkJwt(req,res,next) : void
+attachUserId(req,res,next) : void
delegates match queries
MatchesService
+checkAndCreateMatch(fromUserId,toUserId) : Promise<MatchCheckResult>
+getMyMatches(userId) : Promise<EnrichedMatch[]>
compatibility lookups
reads
swipes/profiles/preferences
emits socket events
Profile
+id: ObjectId
+userId: ObjectId
+name: string?
+age: int?
+gender: string?
+bio: string?
+photos: string[]
+onboardingCompleted: boolean
+city: string?
+hasRoom: boolean
+occupation: string?
+sleepSchedule: string?
+noiseLevel: int?
+guestPolicy: string?
+smoking: string?
+pets: string?
+createdAt: DateTime
+updatedAt: DateTime
owns
0..1
«interface»
ClientToServerEvents
+sendMessage(payload)
+joinRoom(roomId)
+leaveRoom(roomId)
+send_message(payload)
+join(conversationId)
subscribes message events
consumes client contract
«interface»
ServerToClientEvents
+message(payload)
+new_message(payload)
+userOnline(userId)
+userOffline(userId)
Preference
+id: ObjectId
+userId: ObjectId
+genderPreference: string
+minBudget: int
+maxBudget: int
+city: string
+cleanliness: int
+sleepSchedule: int
+smoking: boolean
+drinking: boolean
+pets: boolean
+socialLevel: int
+weightCleanliness: int
+weightSleep: int
+weightHabits: int
+weightSocial: int
+createdAt: DateTime
+updatedAt: DateTime
sets
0..1
ChatSocketGateway
+registerChatSocket(io) : void
emits server contract
reads profiles/preferences
SendMessagePayload
+conversationId: string
+senderId: string
+content: string
validates payload shape
MessagePayload
+id: string
+senderId: string
+content: string
+createdAt: string
+timestamp: string
emits normalized payload
creates
0..*
Swipe
+id: ObjectId
+fromUserId: ObjectId
+toUserId: ObjectId
+action: string
+createdAt: DateTime
1
1
1
fromUserId/toUserId
User
+id: ObjectId
+email: string
+passwordHash: string?
+googleId: string?
+name: string?
+picture: string?
+createdAt: DateTime
+updatedAt: DateTime
participates
0..*
Match
+id: ObjectId
+userAId: ObjectId
+userBId: ObjectId
+createdAt: DateTime
1
1
opens
1
Conversation
+id: ObjectId
+matchId: ObjectId
+createdAt: DateTime
persists outbound chat messages
1
contains
0..*
Message
+id: ObjectId
+conversationId: ObjectId
+senderId: ObjectId
+content: string
+createdAt: DateTime
1
userAId/userBId
senderId
reads matches/conversations
guards protected routes
ChatController
+Openchat(req,res) : Promise<void>
delegates conversation load
ChatService
+getOrCreateConversation(matchId) : Promise<Conversation>
+getMessages(conversationId) : Promise<Message[]>
+sendMessage(conversationId,senderId,content) : Promise<Message>
+validateUserInMatch(matchId,userId) : Promise<boolean>
reads/writes
conversations/messages